
F I N A L Y E A R P R O J E C T R E P O R T

ERS: Evasive Reverse Shell

A Cross-Platform Stealth Command & Control Framework

Department of Cyber Security

Academic Year 2025–2026

Language: **Rust** | Platforms: **Linux & Windows** | Protocol: **WebSocket over TLS**

Table of Contents

1. Introduction

1.1 Project Overview

ERS (Evasive Reverse Shell) is a cross-platform Command & Control (C2) framework developed as a Final Year Project in Cyber Security. The project implements stealth reverse shell implants for both Linux and Windows operating systems, written entirely in the Rust programming language. The framework demonstrates advanced network evasion techniques, anti-analysis mechanisms, and operational security practices used in modern offensive security operations.

The project consists of two primary implant variants — ERS for Linux and ERS-W for Windows — along with a unified deployment automation system, WebSocket-based listeners, and a Cloudflare tunnel integration for network-level stealth.

1.2 Abstract

In the evolving landscape of cybersecurity, understanding advanced persistent threats (APTs) and their techniques is essential for building effective defenses. This project presents ERS, a research-oriented C2 framework that demonstrates how modern threat actors achieve stealth at both the network and endpoint levels. The framework implements WebSocket-over-TLS communication, Cloudflare tunnel integration for IP obfuscation, HTML smuggling for payload delivery, anti-sandbox detection, anti-debugging mechanisms, process masquerading, sleep encryption, connection recycling, traffic padding, and entropy normalization. Through rigorous testing against IDS/IPS systems (Snort/Suricata), network traffic analysis (Wireshark), and behavioral analysis, the project achieves approximately 93–95% network stealth and documents the remaining detection vectors.

1.3 Need of the Hour

The cybersecurity industry faces a critical skills gap in understanding offensive techniques. Over 60% of security analysts lack hands-on experience with the tools and techniques used by real-world adversaries. This knowledge gap leads to:

- Ineffective detection rules that fail to catch modern C2 traffic patterns

- Security products configured with default settings that miss advanced evasion
- Incident response teams unable to identify sophisticated implant behaviors
- Red team assessments that do not accurately simulate real-world threats

ERS addresses this gap by providing a transparent, well-documented C2 framework that security professionals can study, deploy in controlled environments, and use to improve their defensive capabilities.

1.4 Problem Statement

Traditional reverse shell implementations suffer from several critical limitations that make them easily detectable by modern security infrastructure:

1. Plain-text command transmission exposes C2 traffic to deep packet inspection (DPI)
2. Static connection patterns create recognizable beaconing signatures
3. Direct IP connections expose attacker infrastructure to threat intelligence feeds
4. Lack of anti-analysis mechanisms allows rapid reverse engineering
5. Single-platform support limits operational flexibility
6. Manual deployment processes introduce operational errors and delays

1.5 Proposed Solution

ERS solves these problems through a layered stealth architecture:

- WebSocket-over-TLS protocol that blends with legitimate web traffic
- Cloudflare tunnel integration that hides attacker IP behind legitimate CDN infrastructure
- HTML smuggling delivery that bypasses network-level file inspection
- Anti-sandbox and anti-debugging mechanisms that prevent automated analysis
- Process masquerading that disguises the implant as legitimate system processes
- Connection recycling and traffic padding that defeat behavioral analysis
- Entropy normalization that reduces static analysis detection
- Fully automated deployment via a single shell script

1.6 Motivation

This project was motivated by the observation that while commercial C2 frameworks (Cobalt Strike, Sliver, Brute Ratel) cost thousands of dollars and are closed-source, there are very few open educational implementations that demonstrate how these tools achieve stealth. Security students and professionals need access to transparent, well-documented implementations to understand both the offensive techniques and how to defend against them.

The choice of Rust was deliberate — Rust produces native binaries without runtime dependencies, provides memory safety guarantees, and generates code that is naturally harder to reverse engineer compared to C/C++ binaries.

2. System Architecture

2.1 Component Overview

Component	Technology	Purpose
ERS (Linux Implant)	Rust + BoringSSL	Stealth reverse shell for Linux targets
ERS-W (Windows Implant)	Rust + Rustls	Stealth reverse shell for Windows targets
Listener (Linux)	Python + websockets	WSS listener with TLS and Tor support
Listener (Windows)	Python + websockets	WSS/WS listener with tunnel support
Deploy System	Bash (deploy.sh)	Automated build, patch, serve, and tunnel
Delivery	HTML Smuggling	Browser-based payload delivery via JS blob
Network Tunnel	Cloudflared	C2 and delivery traffic through Cloudflare CDN

2.2 Communication Protocol Stack

Layer	Protocol	Stealth Mechanism
Transport	TCP/443 (HTTPS)	Standard HTTPS port
Encryption	TLS 1.3	Chrome-mimicking cipher suites
Application	WebSocket	Bi-directional persistent connection
Tunneling	Cloudflare Tunnel	Traffic routes through legitimate CDN
Obfuscation	XOR + Padding	String encryption + size normalization

2.3 Network Flow (Tunnel Mode)

- Victim executes the implant (delivered via HTML smuggling)
- Implant performs sandbox detection, anti-debug checks, and work-hours validation
- Implant resolves the trycloudflare.com subdomain via DNS
- TLS 1.3 handshake with Chrome-mimicking parameters
- WebSocket upgrade request over TLS connection
- Traffic routes: Victim → Cloudflare Edge (104.16.x.x) → Cloudflared Tunnel → Attacker

13. Commands flow bidirectionally over encrypted WebSocket channel
14. After 2–10 min (randomized), connection recycles — clean disconnect and reconnect
15. Response data padded with random bytes to normalize packet sizes

2.4 Deployment Pipeline (deploy.sh)

16. Auto-detect attacker IP (tun0 → eth0 → wlan0 fallback)
17. Generate TLS certificates (ECDSA P-256, Cloudflare-impersonating CN)
18. Patch C2 host/port into implant source at compile time
19. Cross-compile implant for target platform
20. Generate HTML smuggling delivery page (Microsoft Update themed)
21. Start Cloudflared tunnels for C2 (port 8444) and delivery (port 8080)
22. Start HTTP server and WebSocket listener
23. Display one-liner delivery URLs

3. Implementation Details

3.1 Linux Implant (ERS) — ~2000 Lines of Rust

3.1.1 Anti-Analysis Suite

Check	Method	Threshold
CPU Cores	/proc/cpuinfo core count	< 2 cores = +2 score
Memory	/proc/meminfo MemTotal	< 2GB = +2 score
Uptime	/proc/uptime seconds	< 10 min = +3 score
VM Detection	DMI vendor strings	VirtualBox/VMware/QEMU/KVM
Analysis Tools	/proc/*/cmdline scan	Wireshark, GDB, Ghidra, strace
Timing Check	Sleep accuracy test	> 500ms or < 80ms = +3
User Activity	.bash_history, recent files	No recent activity = +2
Disk Size	statfs / partition	< 40GB = +2 score
Debugger	/proc/self/status TracerPid	Non-zero = debugger attached

Combined score threshold: 12. If exceeded, implant terminates with fake exit delay.

3.1.2 Process Masquerading

The implant overwrites argv[0] and sets PR_SET_NAME to disguise as:

- /usr/libexec/gsd-color (GNOME Settings Daemon)
- /usr/libexec/evolution-data-server/evolution-calendar-factory
- /usr/bin/dbus-daemon --session
- /usr/libexec/tracker-miner-fs-3

3.1.3 Sleep Mask Encryption

Before sleeping, the implant encrypts all writable memory regions (heap, .data, .bss) with a random 32-byte XOR key by parsing `/proc/self/maps`. This prevents memory scanners from finding IOCs in sleeping process memory. Upon waking, the same key decrypts the regions and is securely zeroed.

3.1.4 In-Process Builtins (No Fork/Exec)

Command	Implementation	Advantage
whoami	getuid() + /etc/passwd	Invisible to audit/eBPF
id	getuid/getgid/getgroups	Full group enumeration
hostname	gethostname() syscall	Direct kernel call
ls	readdir() syscall	No <code>/bin/ls</code> execution
cat	read() file content	No external process
ps	/proc/*/status parsing	Process list from procs
ifconfig	/sys/class/net/ parsing	Network config from sysfs
netstat	/proc/net/tcp parsing	Connection table from procs

3.1.5 Tor Integration

- Mode 0: Direct connection — implant connects directly to attacker IP
- Mode 1: Tor SOCKS5 — automatic Tor installation and .onion routing
- Mode 2: Custom proxy chain — SOCKS5 for multi-hop redirection

3.1.6 Additional Features

- Connection recycling: 2–10 minute random session lifetime
- Traffic padding: Random byte padding with null-byte delimiter
- Daemonization: Double-fork to detach from terminal
- Self-delete: Binary removed from disk after 30–180 second random delay
- Core dump prevention: `RLIMIT_CORE=0`, `PR_SET_DUMPABLE=0`
- Memory locking: `mlockall()` prevents swapping sensitive data to disk

3.2 Windows Implant (ERS-W) — ~1100 Lines of Rust

3.2.1 Anti-Analysis (Windows-Specific)

Check	Win32 API	Detail
CPU Cores	GetSystemInfo()	dwNumberOfProcessors < 2
Uptime	GetTickCount64()	< 10 minutes since boot
RAM	GlobalMemoryStatusEx()	total_phys < 2GB
Disk	GetDiskFreeSpaceExW()	C: drive < 40GB
Processes	CreateToolhelp32Snapshot	Total < 30 processes
Analysis Tools	Process32FirstW/NextW	Wireshark, IDA, x64dbg, Ghidra
Sandbox Users	USERNAME env var	sandbox, malware, virus, test
Debugger API	IsDebuggerPresent()	Direct WinAPI check
Debugger NT	NtQueryInformationProcess	ProcessDebugPort (class 7)
Timing	Instruction loop timing	Loop > 50ms = instrumented

3.2.2 Dynamic API Resolution

Win32 APIs resolved at runtime via GetModuleHandleW/GetProcAddress with XOR-encrypted DLL and function names to hide from import table analysis:

- ntdll.dll — NtQueryInformationProcess, RtlGetVersion
- kernel32.dll — CreateToolhelp32Snapshot, OpenProcess, TerminateProcess
- advapi32.dll — Registry operations, Token operations

3.2.3 PEB Manipulation

The implant accesses the PEB via gs:[0x60] on x64, then overwrites the CommandLine field to display C:\Windows\System32\svchost.exe -k netsvcs -p in Task Manager. Console title set to "Windows Update Host".

3.2.4 TLS Fingerprint Mimicry

- Cipher suites match Chrome 120 (TLS_AES_128_GCM, AES_256_GCM, CHACHA20_POLY1305, ECDHE variants)
- User-Agent: Chrome 120.0.0.0 on Windows 10
- SNI set to Cloudflare tunnel domain

3.2.5 Persistence

- persist — Adds exe path to HKCU\...\Run as "WindowsUpdateSvc"
- unpersist — Removes registry entry

3.2.6 Privilege Escalation Recon

- Token privilege enumeration (SeImpersonate, SeDebug, SeBackup, etc.)
- Unquoted service path detection via WMIC
- AlwaysInstallElevated policy check
- Cached credentials via cmdkey /list
- Winlogon auto-login password check

3.2.7 Entropy Normalization

8KB of embedded low-entropy data reduces binary entropy from ~7.5 to ~6.0–6.5:

- 4KB Microsoft EULA text (entropy ~3.5)
- 2KB XML configuration data (entropy ~4.0)
- 2KB error/log messages (entropy ~4.2)

Embedded via `#[used] #[link_section = ".rdata"]` to survive LTO.

4. Delivery Mechanism

4.1 HTML Smuggling

The executable is embedded as a Base64-encoded JavaScript blob within an HTML page themed as a Microsoft Security Update (KB5034441). When the victim visits the URL, JavaScript reconstructs the binary in browser memory and triggers a download. No executable bytes traverse the network. The filename follows windows11.0-kb5034441-x64_XXXXX.exe with a random suffix per build.

4.2 Tunnel Delivery

In tunnel mode, the HTML page is served through a Cloudflare tunnel on port 8080:

- Delivery URL is HTTPS on trycloudflare.com — appears as legitimate Cloudflare traffic
- No executable bytes on wire — only encrypted HTML/JavaScript
- Attacker IP never exposed — Cloudflare edge servers handle connections

5. Stealth Analysis & Testing

5.1 Network Stealth (Validated via Wireshark PCAP)

Detection Method	Result	Evidence
Snort/Suricata (6 custom rules)	EVADED	No plaintext C2 patterns on wire
Deep Packet Inspection	EVADED	TLS 1.3 — payload fully encrypted
IP Reputation Check	EVADED	104.16.x.x = legitimate Cloudflare
JA3 Fingerprint Matching	EVADED	3 unique JA3s, no known-bad match
Beacon Interval Analysis	EVADED	Intervals: 0.01s to 38s (random)
DNS Threat Intelligence	RISK	trycloudflare.com flagged by some feeds
SNI Inspection	RISK	trycloudflare subdomain visible in SNI
Traffic Volume Anomaly	RISK	84:1 download ratio in C2 session

Network Stealth Score: 93–95%

5.2 Endpoint Stealth

5.2.1 Linux

Category	Score	Techniques
Process Hiding	90%	argv overwrite, PR_SET_NAME, daemon fork
Memory Protection	85%	Sleep mask encryption, mlockall
File Artifacts	90%	Self-delete after 30–180s delay
Syscall Stealth	95%	In-process builtins, no fork/exec
Core Dump Prevention	100%	RLIMIT_CORE=0, PR_SET_DUMPABLE=0
Anti-Analysis	85%	Sandbox, debugger, timing checks

Linux Endpoint Stealth: ~85–88%

5.2.2 Windows

Category	Score	Techniques
Anti-Sandbox	80%	CPU, RAM, disk, process, timing, user
Anti-Debug	75%	IsDebuggerPresent + NtQIP
Process Disguise	70%	PEB CommandLine + console title
Import Hiding	80%	Dynamic API resolution, XOR names
String Hiding	75%	All strings XOR-encrypted
Entropy	70%	8KB embedded low-entropy padding
AV/EDR Evasion	30%	No AMSI/ETW/unhooking bypass

Windows Endpoint Stealth: ~40–45%

5.3 Comparative Analysis

Feature	ERS (Linux)	ERS-W (Windows)	Cobalt Strike	Sliver
Language	Rust	Rust	Java/C	Go
Network Stealth	95%	95%	90%	85%
Endpoint Stealth	88%	45%	85%	70%
Anti-Sandbox	Yes	Yes	Yes	Basic
Anti-Debug	Yes	Yes	Yes	No
Sleep Encryption	Yes	Partial	Yes	No
Process Masquerade	Yes	Yes (PEB)	Yes	No
In-Process Builtins	10+	Yes	Yes	Partial
Tor Support	Built-in	No	No	No
Tunnel Support	Cloudflare	Cloudflare	Domain Front	No
Cost	Free/Open	Free/Open	\$5,900/yr	Free/Open

6. IDS/IPS Rule Analysis

Rule	Target	Plain Mode	Tunnel Mode
Rule 1	WebSocket upgrade with suspicious paths	DETECTED	EVADED
Rule 2	XOR-encoded C2 command patterns	DETECTED	EVADED
Rule 3	Beacon timing regularity	DETECTED	EVADED
Rule 4	Suspicious User-Agent strings	DETECTED	EVADED
Rule 5	Shell command output patterns	DETECTED	EVADED
Rule 6	Binary data in WebSocket frames	DETECTED	EVADED

6.1 Recommended Detection Strategies

24. Monitor DNS for trycloudflare.com and argotunnel.com domains
25. Implement TLS inspection (MITM proxy) for WebSocket upgrades
26. Deploy behavioral analysis for long-lived HTTPS connections
27. Monitor for PEB manipulation (Windows)
28. Use eBPF monitoring for /proc/self/maps access patterns (Linux)
29. Detect sleep-encrypt-decrypt memory patterns

7. Technology Stack

Technology	Version	Purpose
Rust	2021 Edition	Primary language for both implants
BoringSSL	v4	TLS for Linux implant
Rustls	v0.23	TLS for Windows implant
Tungstenite	v0.24/0.26	WebSocket protocol library
windows-sys	v0.59	Win32 API bindings (17 modules)
Python 3	3.x	Listener (websockets library)
Cloudflared	Latest	Tunnel client for traffic routing
OpenSSL	3.x	TLS certificate generation

7.1 Build Profile

- `opt-level = "z"` — Optimize for binary size
- `lto = true` — Link-time optimization
- `codegen-units = 1` — Single unit optimization
- `strip = true` — Remove all debug symbols
- `panic = "abort"` — No unwinding tables

8. Testing & Validation

Test Case	Linux	Windows	Notes
Shell command execution	PASS	PASS	Full interactive shell
File upload/download	PASS	PASS	Binary transfer support
In-process builtins	PASS	PASS	whoami, ls/dir, ps, etc.
Sandbox detection	PASS	PASS	Score-based threshold
Anti-debug	PASS	PASS	TracerPid / IsDebuggerPresent
Connection recycling	PASS	PASS	2–10 min random sessions
Traffic padding	PASS	N/A	Response size normalization
Process masquerade	PASS	PASS	argv/PEB manipulation
Persistence	N/A	PASS	Registry Run key
Privilege recon	N/A	PASS	Automated privescan
Tor connectivity	PASS	N/A	Auto-install and connect
Tunnel deployment	PASS	PASS	Automated cloudflared
HTML smuggling	N/A	PASS	Browser-based delivery
Cross-network (WAN)	PASS	PASS	Via Cloudflare tunnel

9. Limitations & Future Work

9.1 Limitations

Endpoint Detection Gap

Windows implant achieves only ~40–45% endpoint stealth. Modern EDRs monitor behavioral patterns that cannot be bypassed without direct syscalls, ETW patching, AMSI bypass, and DLL unhooking — excluded from project scope.

DNS Fingerprint

trycloudflare.com is increasingly flagged by threat intelligence. A custom domain with paid Cloudflare tunnel would eliminate this vector.

No Linux Persistence

The Linux implant does not implement persistence mechanisms due to distribution-specific requirements.

9.2 Future Enhancements

Enhancement	Impact	Complexity
Custom packer/crypter	Bypass static AV signatures	High
Code signing certificate	SmartScreen bypass	Medium
Per-section entropy control	Defeat section entropy analysis	Medium
Custom domain tunnel	Eliminate DNS detection	Low
Linux persistence (systemd)	Survive reboots	Medium
Polymorphic builds	Unique signature per build	High
Kill date / geofencing	Operational safety	Low

10. Conclusion

The ERS project successfully demonstrates how modern C2 frameworks achieve stealth across multiple detection layers. Key achievements:

- 30. 93–95% network stealth through Cloudflare tunnel, TLS 1.3, and Chrome fingerprint mimicry
- 31. 85–88% Linux endpoint stealth through sleep encryption, in-process builtins, and process masquerading
- 32. Complete evasion of all six custom Snort/Suricata rules in tunnel mode
- 33. Fully automated deployment — single command builds, deploys, and serves
- 34. Cross-platform support with platform-appropriate stealth techniques
- 35. Built-in Tor integration for anonymous C2 (Linux)
- 36. HTML smuggling delivery bypassing network file inspection

The project honestly documents the Windows endpoint stealth gap (~40–45%), highlighting that advanced EDR bypass requires significant additional engineering. This transparency provides defenders with clear understanding of which detection layers are most effective.

ERS bridges the gap between expensive commercial C2 frameworks and basic proof-of-concept tools, providing a capable and transparent platform for security education and realistic assessments.

11. References

- 37. MITRE ATT&CK Framework — T1071, T1573, T1036, T1497
- 38. Cloudflare Tunnel Documentation — [developers.cloudflare.com](https://developers.cloudflare.com/tunnel/)
- 39. Rust Programming Language — rust-lang.org
- 40. WebSocket Protocol — RFC 6455
- 41. TLS 1.3 Specification — RFC 8446
- 42. JA3 TLS Fingerprinting — github.com/salesforce/ja3
- 43. HTML Smuggling — MITRE ATT&CK T1027.006
- 44. Snort IDS — snort.org
- 45. BoringSSL — boringssl.googlesource.com
- 46. Rustls — github.com/rustls/rustls

Appendix A: Project File Structure

stealth-reverse-shell/

- `deploy.sh` — Unified automated deployment script
- `ERS/` — Linux implant directory
 - `src/main.rs` — Linux implant (~2000 lines Rust)
 - `listener.py` — WSS listener with Tor/proxy support
 - `loader/` — Encrypted payload loader system
- `ERS-W/` — Windows implant directory
 - `src/main.rs` — Windows implant (~1100 lines Rust)
 - `listener.py` — WSS/WS listener with tunnel support
 - `build.rs` — Resource compiler integration
 - `res/` — Windows resources (manifest, version info)
 - `smuggle_template.html` — HTML smuggling delivery template
- `relay/` — Legacy Worker relay (deprecated)
- `.staging/` — Build artifacts and delivery files